Multimedia Signal Processing - 2nd Module

July, 12 2016

1. **Question (8 pts)**
   Consider a continuous source X that emits symbols following a probability distribution function (p.d.f.) $p_X(x)$.
   (a) Explain how to design the Lloyd-Max quantizer. Consider $M = 3$ quantization bins and use the thresholds $\{-\infty, a_2, a_3, +\infty\}$ as initialization.
   (b) Consider the following series of symbols emitted by X

   $$x = \{-0.9,\ 0.1,\ -0.8,\ -0.7,\ 0.7,\ -0.6,\ -0.2,\ 0.1,\ 0.7,\ -0.2,\ 1.0\}.$$

   Design a Lloyd-Max quantizer tailored to these symbols using $M = 3$ bins and the thresholds $\{-\infty, -0.5, 0.5, +\infty\}$ as initialization. Stop after one complete iteration.
   (c) How many iterations must be performed to reach the algorithm's convergence?
   (d) Quantize x using the Lloyd-Max quantizer designed at point (b), thus obtaining $\tilde{x}$.
   (e) Compute the entropy of the source representing $\tilde{x}$ estimating the probability mass function (p.m.f.) directly from $\tilde{x}$.
   (f) Derive an Huffman code to transmit $\tilde{x}$. Represent using this code a string of four symbols of your choice.

   Solution
   (a) See lecture slides.
   (b) First we compute the probability of each symbol according to their occurrences:
   $p_{-0.9} = 1/11$, $p_{-0.8} = 1/11$, $p_{-0.7} = 1/11$, $p_{-0.6} = 1/11$, $p_{-0.2} = 2/11$, $p_{0.1} = 2/11$, $p_{0.7} = 2/11$, $p_{1.0} = 1/11$.
   We now compute the quantizer levels in each interval $[a_k, a_{k+1}]$ by summing the symbols multiplied by their probability, and normalizing everything according to the sum of their probabilities:
   $y_1 = \frac{\frac{1}{11}(-0.9-0.8-0.7-0.6)}{\frac{4}{11}} = -0.75$; $y_2 = \frac{\frac{2}{11}(-0.2+0.1)}{\frac{4}{11}} = -0.05$; $y_3 = \frac{1\frac{1}{11}+0.7\frac{2}{11}}{\frac{3}{11}} = 0.8$.
   Finally, we update the thresholds:
   $a_1 = -\infty$; $a_2 = \frac{y_2+y_1}{2} = -0.4$; $a_3 = \frac{y_3+y_2}{2} = 0.375$; $a_4 = +\infty$.
   (c) The algorithm already reached convergence. Indeed, if we compute another iteration, we obtain the same y and a values.
   (d) $\tilde{x} = \{-0.75,\ -0.05,\ -0.75,\ -0.75,\ 0.8,\ -0.75,\ -0.05,\ -0.05,\ 0.8,\ -0.05,\ 0.8\}$.
   (e) Let us compute the probability of each quantized symbol observing their occurrences in $\tilde{x}$:
   $p_{-0.75} = 4/11$, $p_{-0.05} = 4/11$, $p_{0.8} = 3/11$.
   The entropy is: $H = -[\frac{4}{11}\log_2(\frac{4}{11}) + \frac{4}{11}\log_2(\frac{4}{11}) + \frac{3}{11}\log_2(\frac{3}{11})] = 1.5726$ bps.

(f) A possible Huffman code is:

Symbol	p.m.f.	Code	Length
0.8	3/11	11	2
−0.05	4/11	10	2
−0.75	4/11	0	1

The sequence $\{-0.75,\ -0.05,\ -0.75,\ -0.75\}$ is encoded as 01000.

2. **Question (5 pts)**
   (a) Illustrate the block diagram of a hybrid DCT/DPCM video encoder.
   (b) Illustrate the block diagram of a hybrid DCT/DPCM video decoder.

   Solution See lecture slides.

3. **Question (5 pts)**
   (a) Illustrate the principles of transform coding.
   (b) Provide its geometric interpretation.

   Solution See lecture slides.

4. **Questions (5 pts - each answer can be either TRUE or FALSE)**
   (a) Consider an image coding scheme.
   - [T] [F] Each color channel in the RGB space is completely independent on natural images.
     FALSE. $H(R,G,B)$ is typically less than $H(R)+H(G)+H(B)$.
   - [T] [F] It is better to sub-sample chroma components than luminance component.
     TRUE.
   - [T] [F] Neighboring pixels typically share some form of correlation.
     TRUE.
   - [T] [F] KLT is used for JPEG compression.
     FALSE. DCT is used.

   (b) Consider an audio coding scheme.
   - [T] [F] The concept of critical bandwidth is only used for speech coding.
     FALSE. It is exploited in general audio coding and MPEG standards.
   - [T] [F] MPEG-1 psychoacoustic model exploits both tonal and noise maskers.
     TRUE.
   - [T] [F] The absolute threshold of hearing has a minimum around 3 or 4 kHz.
     TRUE.

   (c) Consider two independent discrete memoryless sources X and Y.
   - [T] [F] $H(X|Y)+H(Y)=H(X,Y)$.
     TRUE. As X and Y are independent $H(X|Y)=H(X)$.
   - [T] [F] The average length of the Huffman code L_X is always greater than $H(X)$.
     FALSE. It can be equal.
   - [T] [F] The maximum distortion achievable quantizing X is given by the source variance σ_X^2.
     FALSE. It can be anything, there is no limit to the maximum distortion.

5. **Question (Matlab - 10pts)**
   (a) Load the file `table_tennis.mat`, which contains two frames of a video sequence (i.e., `table_tennis(:,:,1)` and `table_tennis(:,:,2)`). Display both frames on the screen and describe what they represent.

   (b) Consider each frame as a monodimensional realization generated by a discrete memoryless source. Compute the entropy of each frame separately.

   (c) Consider splitting each frame into 8×8 non-overlapping blocks. For each block of the first frame, compute a motion vector (`MV = [x y]`) by minimizing the mean squared error (MSE) with a block of the second frame. Only consider a neighborhood of ± 1 blocks with respect to the reference position (i.e., at most 9 comparisons per reference block).

   (d) Compute the entropy of the MVs considering: i) the horizontal component only (i.e., `x`); ii) the vertical component only (i.e., `y`); iii) both components jointly (i.e., `[x y]`).

   (e) Given the MVs, compute and display the displaced frame difference (DFD).

   (f) As in point (b) compute the entropy of the DFD.

(g) Considering that you work with an optimum code, which is the minimum number of bits needed to transmit both frames: i) when each one is transmitted separately; ii) when you transmit only one frame, the DFD and the MVs.

Solution

```
%clc, clear all, close all


%% Load and display video frames
load('table_tennis.mat')
frame_1 = table_tennis(:,:,1);
frame_2 = table_tennis(:,:,2);

figure(1)
subplot(131), imshow(uint8(frame_1)), title('Frame 1')
subplot(132), imshow(uint8(frame_2)), title('Frame 2')


%% Entropy of each frame
p_f1 = hist(frame_1(:), unique(frame_1(:)));
p_f1 = p_f1./sum(p_f1);
H_f1 = -sum(p_f1.*log2(p_f1));

p_f2 = hist(frame_2(:), unique(frame_2(:)));
p_f2 = p_f2./sum(p_f2);
H_f2 = -sum(p_f2.*log2(p_f2));


%% Motion Estimation
%%% Parameters
B = 8;
[H, W] = size(frame_1);

%%% Count blocks
N_H = H/B;
N_W = W/B;

%%% Initialize motion vectors
mv_x = zeros(N_H, N_W);
mv_y = zeros(N_H, N_W);

%%% Initialize DFD
dfd = zeros(size(frame_1));

%%% Comupute motion vectors
for h1 = 1:N_H
    for w1 = 1:N_W

        %%% Select reference block
        block_1 = frame_1((h1-1)*B+1:h1*B, (w1-1)*B+1:w1*B);

        %%% Compute MSE with blocks from the other frames
        mse = inf*ones(3,3);
        for h2 = -1:1
            for w2 = -1:1
                if (h1+h2-1)*B+1 > 0 && (h1+h2)*B <= H && (w1+w2-1)*B+1 > 0 && (w1+w2)*B <= W
                    block_2 = frame_2((h1+h2-1)*B+1:(h1+h2)*B, (w1+w2-1)*B+1:(w1+w2)*B);
                    mse(h2+2, w2+2) = mean((block_1(:) - block_2(:)).^2);
                end
            end
        end

        %%% Find minimum MSE
        min_mse = min(mse(:));
        [dy, dx] = find(mse == min_mse);

        %%% Save motion vectors
        mv_x(h1, w1) = dx-2;
        mv_y(h1, w1) = dy-2;

        if dx ~= 2
            pippo = 1;
        end

        %%% Compute DFD
        h2 = mv_y(h1, w1);
        w2 = mv_x(h1, w1);
        best_block = frame_2((h1+h2-1)*B+1:(h1+h2)*B, (w1+w2-1)*B+1:(w1+w2)*B);
        dfd((h1-1)*B+1:h1*B, (w1-1)*B+1:w1*B) = block_1 - best_block;

    end
end

%%% Display DFD
subplot(133), imagesc(dfd), title('DFD'), axis image, axis off
```

```
%% Entropy of DFD
p_dfd = hist(dfd(:), unique(dfd(:)));
p_dfd = p_dfd./sum(p_dfd);
H_dfd = -sum(p_dfd.*log2(p_dfd));


%% Compute MV Entropy
%%% Entropy on X component
p_x = hist(mv_x(:), [-1, 0, 1]);
p_x = p_x/sum(p_x);
p_x(p_x == 0) = [];
H_x = -sum(p_x.*log2(p_x));

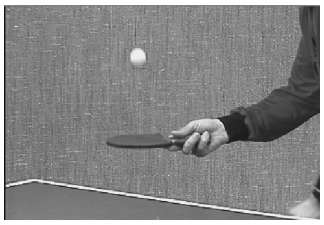
%%% Entropy on Y component
p_y = hist(mv_y(:), [-1, 0, 1]);
p_y = p_y/sum(p_y);
p_y(p_y == 0) = [];
H_y = -sum(p_y.*log2(p_y));

%%% Joint entropy
ctrs{1} = [-1, 0, 1];
ctrs{2} = [-1, 0, 1];
mv_xy = [mv_x(:), mv_y(:)];
p_xy = hist3(mv_xy, ctrs);
p_xy = p_xy(:)./sum(p_xy(:));
p_xy(p_xy==0) = [];
H_xy = -sum(p_xy.*log2(p_xy));

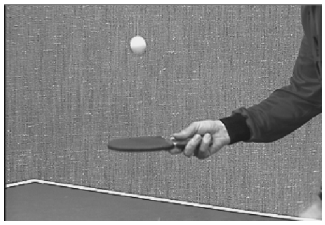
%%% Comparison
disp(['H_x + H_y = ', num2str(H_x + H_y)])
disp(['H_xy      = ', num2str(H_xy)])


%% Amount of used bits
bits_1 = H_f1*H*W + H_f2*H*W
bits_2 = H_f1*H*W + H_dfd*H*W + H_xy*N_H*N_W
```

Frame 1



Frame 2



DFD

